How Sealroom works

Two law firms find out whether they share clients without either one showing its client list to anyone. Then the firm's AI assistant works with the conflicted clients walled off. Every step leaves a receipt anyone can check.

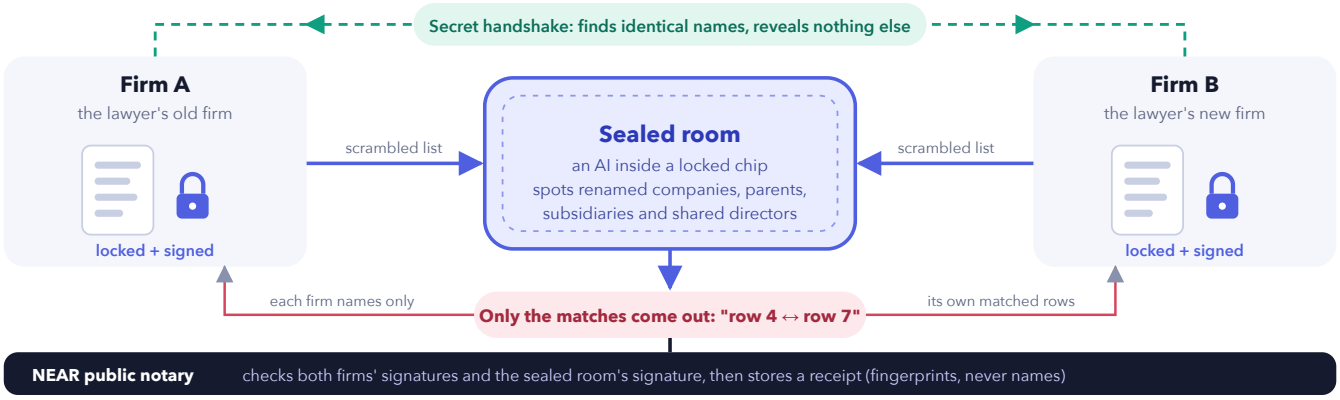
The problem

When a lawyer moves firms, both firms must check for **conflicts of interest**: is the lawyer's old client someone the new firm is suing, or acting for? Today one side hands its **confidential client list** to the other, or to a vendor, and simply has to trust them.

The idea

Each firm locks its list in its own browser and drops it into a **sealed room**: a computer chip that even its operator cannot look inside. Only the matches come out. A **public notary** (the NEAR blockchain) checks everyone's signatures and keeps a receipt.

1 The conflict check



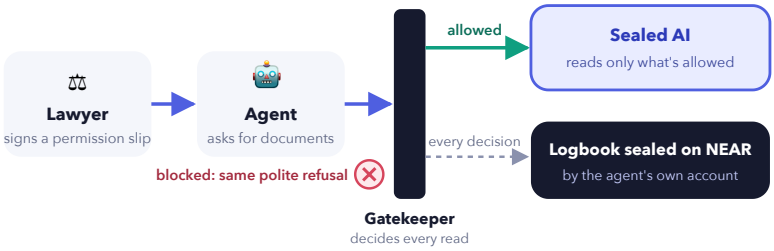
1. **Check the room is genuine**
   Intel's certificate proves the chip is real, sealed and unmodified.
2. **Lock and sign**
   Each firm scrambles its list in its own browser and signs it with its registered key.
3. **Secret handshake**
   Exact name matches ("Vale Doce S.A." = "Vale Doce") are found by maths alone.
4. **The sealed AI looks deeper**
   It answers with row numbers only, never names, so nothing else can leak.
5. **The notary records it**
   NEAR refuses the record unless every signature checks out.
6. **Ranked result**
   "Old client vs. the new firm's opponent" comes first, then cases to review.

2 Everyone has an ID card

🏛 **Firm**	Registers its own signing key. A list it didn't sign is rejected.
⚖ **Lawyer**	Own NEAR account, added by the firm. Signs every AI session they start.
🤖 **AI agent**	Owned by a partner, locked to one AI model and one set of instructions. Its key can do exactly one thing.

Like a building pass, with NEAR as the front desk that checks it every time. A firm can cancel a pass at any moment.

3 The AI assistant, with walls



Conflicted clients from the check become walls. Every blocked read gets the same refusal, so the AI can't work out who the firm acts for.

4 Why you don't have to trust us

- ✓ No list ever leaves a firm unscrambled. Our server only sees row numbers.
- ✓ Each firm can later prove exactly which list it submitted, without showing it.
- ✓ Tampering with the result, the AI's instructions or a reply is detected.
- ✓ Every result comes with a **proof pack** anyone can replay to get the same answer.
- ✓ The notary rejects anything with a missing or forged signature.
- ✓ The AI agent can't swap its model or instructions, or act without a lawyer's signature.

Sealed room (enclave)
A sealed processor whose memory is encrypted. Intel signs a certificate saying exactly what runs inside.

NEAR
A public blockchain. Here it is a notary: it stores receipts and checks signatures, but never sees client names.

Signature
A digital stamp only the key's owner can make, and anyone can check against their registered ID.

Fingerprint
A short code computed from a list. It proves which list was used but can't be turned back into the list.

Built on NEAR

Every NEAR building block Sealroom uses, what it does here, and where to find it in the code. The NEAR AI Cloud parts keep the lists private; the NEAR Protocol parts make every claim checkable.

WHAT WE USED	WHAT IT DOES IN SEALROOM	DETAILS
NEAR AI Cloud: the sealed room		
Private inference in a TEE	Runs the conflict model on lists nobody else can read	`z-ai/glm-5.3-flash` on NEAR AI Cloud, inside an Intel TDX confidential VM with GPU TEE
Attestation API	Proves the model runs in a genuine, non-debug enclave before any list is sent	`/v1/attestation/report` with a fresh nonce; quote checked against Intel PCS with `dcap-qvl` ; the model key and nonce must be bound in the quote (`attest.py`)
End-to-end encryption	Each firm encrypts its own list in its browser, straight to the enclave's key	Protocol v2: X25519 + HKDF-SHA256 + XChaCha20-Poly1305, a fresh ephemeral key per field (`e2ee.py` , `sealcrypto.js`)
Signed responses	The enclave signs every request and reply, so a verdict can't be altered after the fact	`/v1/signature/{chat_id}` , ed25519, checked against the attested key off-chain and on-chain (`nearai.py`)
NEAR Protocol: the notary		
Rust smart contract	Identity registry, conflict-check records, agent session seals	`near-sdk` 5, built with `cargo-near` , deployed to `v2.sealroom-64716.testnet` (`contract/src/lib.rs`)
`env::ed25519_verify`	The contract itself checks both firms' signatures and up to 48 enclave signatures per check	About 18 Tgas for a full check; a record with a missing or forged signature is refused
`env::sha256`	Stores one root over every signed enclave call	Lets anyone show a call was dropped or swapped
Named accounts and sub-accounts	Each firm is an account; each lawyer is a real sub-account with their own key	`jane.sr-firm-south-64716.testnet` , created by the firm and deactivatable by it
Function-call access keys	The research agent's only key can call `seal_session` and nothing else	0.5 NEAR gas allowance; the temporary full-access key is deleted during `sealroom setup`
`predecessor_account_id`	Who may do what: a firm manages only its own members and agents; an agent seals only its own sessions	The contract re-checks the lawyer's signature, the model and the prompt hash on every seal
Contract logs	Explain a refusal even when testnet reports a bare trap	The contract logs `sealroom: <reason>` before it panics; the CLI reads it back
Tooling		
near-cli-rs	Deploys, sets up accounts and keys, and signs the server's transactions	0.22, legacy keychain, so keys never pass through our code (`chain.py` , `setup.py`)
FastNEAR RPC	Reads firms, members, agents and records; fetches refusal logs	`test.rpc.fastnear.com`
NearBlocks	Every verdict and session in the app links to its transaction	`testnet.nearblocks.io`

On testnet now

Contract v2.sealroom-64716.testnet
Operator sealroom-64716.testnet
Firm (incoming) sr-firm-north-64716.testnet · Northgate & Pryce LLP
Firm (existing) sr-firm-south-64716.testnet · Southwell Carrow LLP
Lawyers jane. and marcus.sr-firm-south-64716.testnet
Research agent research.sr-firm-south-64716.testnet

Not covered yet

- ✓ The contract can't verify an Intel quote itself. It checks the enclave's signatures, and the proof bundle carries the raw attestation so anyone can check the key.
- ✓ We don't replay RTMR3 against the app compose or check NVIDIA GPU attestation through NRAS.
- ✓ The access rules run on our server. Running it as a Shade Agent inside a TEE is the next step.